

RepoScout

A Field Guide to the Codebase

How every piece works, and why it was built that way

A reference iOS application demonstrating 2026 best practices

SwiftUI · Swift 6 concurrency (SE-0461) · Combine · SwiftData · Swift Testing · XCUITest with BDD activities

Project: `testExample.xcodeproj` (product branded RepoScout, bundle ID `work.timmaher.RepoScout`)

Requires Xcode 26.2+ · iOS 26.2 deployment target · iPhone · English + German · Zero third-party dependencies
· MIT licensed

Second edition – revised after a three-way external audit (idiom currency, concurrency correctness, shipping hygiene)

August 2026

Contents

1. What RepoScout Is
2. Running the App and the Tests
3. The Map: Project Structure
4. The Life of a Search – an End-to-End Trace
5. The Model Layer: Three Shapes for One Repository
6. The Service Layer: a Protocol, Two Clients, and Typed Errors
7. LoadState: Stale-While-Revalidate as a Type
8. SearchViewModel: the App's Only View Model
9. The Views: Where Logic Was Deliberately Left Out
10. Persistence: SwiftData, @Query, and the Store Split
11. The Composition Root: Dependency Injection Without Magic
12. The Concurrency Model: Swift 6, SE-0461, and What "async" Actually Means Now
13. Combine's Role in 2026 – the Honest Boundary
14. Unit Testing with Swift Testing
15. UI Testing: BDD Scenarios over XCTContext Activities
16. Localization and Accessibility
17. Shipping Hygiene: the Parts That Aren't Code
18. Design Decisions, Trade-offs, and Known Edges

1 What RepoScout Is

RepoScout is a small, complete iOS app: you search GitHub's public repository API, browse results, open a detail screen, and save favorites that persist locally. That functionality is deliberately modest – the app exists to be *read*, not shipped. Every file is written to demonstrate how a professional iOS team would build this in 2026, and the doc comments inside the source explain the *why* behind each pattern, not just the what.

How to read this guide: Section 4 walks one keystroke through the entire system – read it first, because every later section deepens one hop of that journey. Sections 5–11 cover the design decisions worth stealing: the DTO/domain split, `LoadState` with stale-while-revalidate, protocol-based DI, the `@Query` -for-reads/store-for-writes split. Section 12 is the concurrency chapter – it explains SE-0461, the 2025 change to what `async` means, which most material written before it silently gets wrong. Sections 14–17 cover the testing and shipping practices that separate a demo from a product.

This is the second edition. The codebase was subjected to a three-way external audit – SwiftUI idiom currency, hostile concurrency review, and professional-completeness review – and every confirmed finding was fixed. Several sections below teach lessons that exist *because* the audit caught the first version getting them wrong; those are marked, because "here is the mistake and why it was wrong" is often the most durable form of the lesson.

Naming note. The Xcode project, targets, and schemes are still named `testExample`; the product is branded RepoScout via `INFOPLIST_KEY_CFBundleDisplayName`, a real app icon, and the bundle ID `work.timmaher.RepoScout`. Renaming a `pbxproj` is high-churn, low-value work; product identity and project naming are independent, and leaving them different demonstrates that.

2 Running the App and the Tests

Open `testExample/testExample.xcodeproj` in Xcode 26.2 or later and run the `testExample` scheme. The scheme is **shared** (committed under `xcshareddata/xcschemes/`), so the commands below work on a fresh clone and on CI – an easy thing to get wrong, because Xcode's auto-created schemes are per-user and untracked, which makes documented commands work for the author and fail for everyone else.

```
# Unit tests only (Swift Testing) – fast
xcodebuild test -project testExample.xcodeproj -scheme testExample \
  -destination 'platform=iOS Simulator,name=iPhone 17 Pro' \
  -only-testing:testExampleTests

# UI tests (XCUITest, BDD-style) – boots a simulator, slower
xcodebuild test -project testExample.xcodeproj -scheme testExample \
  -destination 'platform=iOS Simulator,name=iPhone 17 Pro' \
  -only-testing:testExampleUITests

# Everything
xcodebuild test -project testExample.xcodeproj -scheme testExample \
  -destination 'platform=iOS Simulator,name=iPhone 17 Pro'
```

Substitute any installed simulator for the `name=` value. The UI tests never touch the network: they launch the app with debug-only arguments (Section 11) that swap in a deterministic mock client and an in-memory database, so the whole suite runs offline and produces identical results every time. To see the German localization, run the app with App Language set to German in the scheme editor, or pass `-testLanguage de` to a test run.

3 The Map: Project Structure

Path	Responsibility
testExample/RepoScoutApp.swift	@main entry point; owns AppDependencies
testExample/Models/	Repo (domain model) and GitHubSearchResponse / RepoDTO (network DTOs + the one mapping seam)
testExample/Services/	GitHubClient protocol, GitHubClientError , LiveGitHubClient (@concurrent , URLSession), MockGitHubClient (DEBUG-only actor)
testExample/Support/	LoadState , AppDependencies composition root, PreviewSupport , shared Logger
testExample/ViewModels/	SearchViewModel – the app's only view model
testExample/Persistence/	FavoriteRepo (@Model) and FavoritesStore (write-side repository)
testExample/Views/	RootView (TabView), Search/ , Detail/ , Favorites/
testExample/Localizable.xcstrings	String Catalog – every user-visible string, English + full German, plural variations, translator comments
testExample/PrivacyInfo.xcprivacy	Privacy manifest (Section 17)
testExampleTests/	Swift Testing suites + TestSupport.swift (actor test doubles, StubURLProtocol , polling helpers)
testExampleUITests/	BDD helpers, screen objects, scenario files incl. accessibility audits
README.md , ARCHITECTURE.md , LICENSE	Quick start, the guided tour, MIT license
docs/process-archive/	Point-in-time build planning artifacts, kept with disclaimers; superseded by the code

The project uses Xcode's *synchronized folder groups* (`PBXFileSystemSynchronizedRootGroup`): any file placed in a target's folder on disk automatically joins that target. Nobody edits `project.pbxproj` to add files anymore.

4 The Life of a Search – an End-to-End Trace

Everything in this app can be understood by following one keystroke. Suppose the user types `"swift"` into the search field.

```

keystroke → .searchable binding → SearchViewModel.searchText (didSet)
→ querySubject.send() → Combine: debounce(300ms, main) → filter(≠ lastDispatchedQuery)
→ sink: record query, cancel old Task, start new Task → await search(matching:)
→ state = .loading (first load) or .loaded(stale, isRefreshing: true) (refinement)
→ await client.searchRepositories() (@concurrent – leaves the main actor here)
→ URLRequest (Accept, User-Agent, 15s timeout) → JSON → RepoDTO → Repo
→ state = .loaded([Repo], isRefreshing: false) → @Observable tracking invalidates
SearchView
→ switch state → List of RepoRowView (exhaustive – compiler-checked)

```

Hop by hop:

1. **The binding.** `SearchView` declares `.searchable(text: $viewModel.searchText, ...)` with autocapitalization and autocorrection disabled – repository names are identifiers, not prose, and iOS's defaults would "fix" `vapor` into `vapour`. Pressing Return calls `submitImmediately()`, bypassing the debounce the user has already outrun.
2. **Into the stream.** `searchText`'s `didSet` forwards every change into a Combine `PassthroughSubject`. This is the moment a value becomes an *event in a stream*.
3. **The pipeline.** `debounce(for: .milliseconds(300), scheduler: DispatchQueue.main)` waits for a pause in typing; then a `filter` drops the value if it equals `lastDispatchedQuery`. Why a filter and not `removeDuplicates()` is a lesson in itself – Section 8.
4. **Stream, meet structured concurrency.** The `sink` records the query, cancels any in-flight search `Task`, and starts a new one – the boundary where the Combine world hands off to `async/await`.
5. **State respects what's on screen.** A first load sets `.loading` (full-screen spinner). A *refinement* of existing results sets `.loaded(existing, isRefreshing: true)` – the stale results stay visible with a small indicator, instead of flashing away on every keystroke. Section 7.
6. **The network, behind a protocol – and off the main actor.** The view model only knows `any GitHubClient`. The production implementation is marked `@concurrent`, which is what actually moves the request and the JSON decode off the caller's executor – under SE-0461, `async` alone would not. Section 12.
7. **JSON becomes domain.** The response decodes into `RepoDTO` (GitHub's shape) and maps into `Repo` (our shape) via `Repo.init(dto:)` – the single seam where the API's vocabulary ends.
8. **The UI is a function of state.** `SearchView.content` is a `switch` over `LoadState` with no `default`. Add a case someday and the compiler points at every view that needs updating.

5 The Model Layer: Three Shapes for One Repository

The same conceptual thing – "a GitHub repository" – exists as three distinct Swift types, on purpose:

Type	File	Shape belongs to	Changes when...
RepoDTO	Models/GitHubSearchResponse.swift	GitHub's API	GitHub changes their JSON
Repo	Models/Repo.swift	This app's domain	The app's needs change
FavoriteRepo	Persistence/FavoriteRepo.swift	The on-disk schema	A migration is written

These three evolve on different clocks, which is why they are not one type even though the fields rhyme. Each pair is connected by exactly one conversion: `Repo.init(dto:)`, `FavoriteRepo.init(repo:)`, and `FavoriteRepo.asRepo`. When something changes, one seam moves.

`Repo` is a `nonisolated struct` conforming to `Identifiable` (so `List` renders it), `Hashable` (so it can be a type-safe navigation value), and `Sendable` (trivially, as an immutable value type – which is what lets it cross the executor boundary the `@concurrent` client creates). The DTOs mirror GitHub's `snake_case` JSON with explicit `CodingKeys` rather than a global `convertFromSnakeCase` strategy: a few lines of boilerplate in exchange for greppability – search for `full_name` and you land exactly on the mapping.

6 The Service Layer: a Protocol, Two Clients, and Typed Errors

The seam that makes everything testable

```
nonisolated protocol GitHubClient: Sendable {
    func searchRepositories(matching query: String) async throws -> [Repo]
}
```

One method, `async`, returning domain models. Everything downstream – view model, views, unit tests, UI tests, previews – is oblivious to which implementation is running. Five exist: the production client, the `DEBUG` mock, and three actor-based test doubles.

The live client: honest HTTP, honest errors

`LiveGitHubClient` builds a real `URLRequest`: a 15-second timeout, `Accept: application/vnd.github+json` to pin the response shape, and a `User-Agent` – which the GitHub API requires; anonymous requests are rejected. The status handling is where the first edition got caught being lazy, and the fixes are each a small lesson in reading an API's actual contract:

```

switch http.statusCode {
case 200..300: // any 2xx is success – matching only 200 turns a valid 203 into an "error"
    break
case 400, 422: // GitHub answers a malformed/empty query with 422 – THIS is .invalidQuery's
    throw GitHubClientError.invalidQuery // real producer (the URL-building guard can't fail)
case 403: // 403 is overloaded: rate limit AND plain forbidden. Only claim "give it
    // a minute" when GitHub's own headers say so; otherwise it's a server error.
    if http.value(forHTTPHeaderField: "x-ratelimit-remaining") == "0"
        || http.value(forHTTPHeaderField: "Retry-After") != nil {
        throw GitHubClientError.rateLimited
    }
    throw GitHubClientError.server(statusCode: 403)
case 429:
    throw GitHubClientError.rateLimited
default:
    throw GitHubClientError.server(statusCode: http.statusCode)
}

```

Cancellation gets first-class treatment: `URLSession` reports a cancelled task as `URLError(.cancelled)` (it never throws `CancellationError` itself), and the client normalizes that into `CancellationError` so callers handle cancellation in one shape. Without this, a superseded search would masquerade as "Couldn't reach GitHub" – routine typing would produce scary error copy. The entire mapping table is pinned by `URLProtocol`-stub tests (Section 14), including the cancellation path.

The function itself is marked `@concurrent` – the load-bearing keyword explained in Section 12.

The mock client

`MockGitHubClient` is a DEBUG-only *actor* (it counts calls, and an actor is the default answer for mutable state) living in the app target, because previews and UI-test launches execute the real app binary. Four scripted scenarios, selected by an environment variable: `success`, `searchError`, `emptyResults`, and `searchErrorOnce` (fails the first call, succeeds after – which is what lets a UI test prove the Retry button actually recovers). A 300ms artificial delay keeps loading states visible; it sits before the scenario switch deliberately, so a cancelled mock call throws `CancellationError` like the real client would.

7 LoadState: Stale-While-Revalidate as a Type

```

nonisolated enum LoadState<Value: Sendable & Equatable>: Equatable, Sendable {
    case idle
    case loading
    case loaded(Value, isRefreshing: Bool)
    case failed(message: String)
}

```

The core argument is unchanged from every good writeup of this pattern: `isLoading: Bool + items: [Repo] + error: Error?` gives eight combinations, most of them nonsense, and every view must

defensively decide what `isLoading && error != nil` means. A closed enum means exactly one state exists and the compiler guarantees every case has a screen.

The interesting part is `isRefreshing` – and it's there because the audit caught the first edition walking into the *standard critique* of naive four-case load enums: with only `.loading` and `.loaded`, refining a query must either blank the results to a spinner (the screen flickers on every debounced keystroke – which is what v1 shipped) or lie about a request being in flight. "Results on screen while a refinement is loading" is a real, necessary state; a type that claims to make illegal states unrepresentable had made a *legal* one unrepresentable. The `Bool` lives *inside* `loaded` rather than as a sibling property, so "refreshing with nothing to refresh" remains impossible. In the UI: a first load shows the full-screen spinner; a refinement keeps the list and shows a mini-spinner in the footer.

`failed` carries a display-ready `String`, not the `Error` – errors aren't `Equatable`, and by the time state reaches a view the only remaining question is "what do we tell the user?"

8 SearchViewModel: the App's Only View Model

RepoScout has exactly one view model, and that's a lesson in itself: a view model must earn its keep with real orchestration logic. The search screen has plenty; the detail and favorites screens have none, so they don't get one. It's an `@Observable final class` – views track exactly the properties they read – `MainActor`-isolated by the project's default isolation, with all state `private(set)` except the bindable `searchText`.

Why `filter`, not `removeDuplicates()`

The first edition used `removeDuplicates()` after the debounce, and the audit found the bug that choice hides: **`removeDuplicates()` dedups on the input, and input-dedup is only sound when the operation is deterministic.** A network search is not – the same text can fail once and succeed next time. With `removeDuplicates()`, a search that failed became literally unrepeatable by typing: the operator remembered the failed query forever, so retyping it was swallowed, and only the Retry button escaped. The replacement dedups against an explicit `lastDispatchedQuery` that the *outcome* controls:

```
querySubject
    .debounce(for: debounceInterval, scheduler: DispatchQueue.main) // load-bearing – see §12
    .filter { @MainActor [weak self] query in query != self?.lastDispatchedQuery }
    .sink { @MainActor [weak self] query in
        guard let self else { return }
        lastDispatchedQuery = query
        searchTask?.cancel()
        searchTask = Task { await self.search(matching: query) }
    }
    .store(in: &cancellables)
```

- After **success**, the query stays recorded – identical text doesn't re-search.
- After **failure**, `search(matching:)` clears it – retyping the same text retries.

- On **Retry or Return**, `submitImmediately()` records the query *before* dispatching – so a debounce emission already in flight gets filtered instead of racing a second identical request onto the wire (the other half of the audit's finding: retry racing a pending debounce used to double-fire).

Supersession: why a stale search can't win

The sink cancels the previous `searchTask` before starting a new one, and `search(matching:)` re-checks `Task.isCancelled` after the `await`, before writing state. The comment in the source now states the exact invariant that makes this airtight rather than a TOCTOU: *self is MainActor-isolated, the only code that can cancel searchTask runs on the main actor, and there is no suspension point between the guard and the writes – so a racing cancellation lands entirely before the check or entirely after the writes*. That reasoning is load-bearing; lift the pattern into a class where cancellation can arrive from a background context and it stops being airtight. A dedicated test drives a cancelled task all the way through a *successful* response and proves the stale result never lands.

The ticker: scoped, seeded, and confined to a leaf

The results footer shows "Updated N seconds ago", driven by a Combine `Timer.publish`. Three deliberate design points, each one a fix for a real audit finding:

- **Scoped lifecycle** – `startTicker()` / `stopTicker()` are called from the results list's `.onAppear` / `.onDisappear`, so the publisher runs only while the UI that displays its output is on screen. An init-started timer on this app-lifetime object would fire forever, on every tab.
- **Seeded start** – `startTicker()` sets `now = .now` before subscribing, because `Timer.publish`'s first emission is a full second away. Without the seed, returning to the tab after five minutes displayed "Updated 0s ago" (a negative interval clamped by `max(0,...)` – the clamp was silently hiding the bug) until the first tick corrected it.
- **Leaf-view invalidation confinement** – the footer is its own private `View` struct, `LastRefreshedFooter`, and this is the file's sharpest lesson: `@Observable` attributes property reads to whichever `body` performs them. Inline the footer text into `SearchView` and the once-per-second `now` read belongs to `SearchView.body` – the entire screen, `NavigationStack`, `List` and all, re-evaluates every second (which is exactly what v1 did while its doc comment explained why that would be bad). As a leaf, the per-second dependency belongs to a view whose body is two `Text` s.

The footer is also `.accessibilityHidden(true)` – a VoiceOver element whose label changes every second is hostile, and the text is ambient status, not content – and `.monospacedDigit()`, so the digits don't jitter as widths change. The visible string is "Updated 42 seconds ago", spelled out rather than "42s": the catalog carries plural variations, and a language that inflects the noun needs a noun to inflect.

9 The Views: Where Logic Was Deliberately Left Out

RootView

A `TabView` using the modern `Tab` builder, with the search tab declared `role: .search` – one keyword that opts into iOS 26's dedicated search-tab placement and behaviors. The `SearchViewModel` arrives from the composition root; nothing is a global.

SearchView

A pure function of `viewModel.state` :

State	UI
<code>.idle</code>	<code>ContentUnavailableView</code> – "Search GitHub" invitation
<code>.loading</code>	Full-screen <code>ProgressView</code> – <i>first</i> loads only
<code>.loaded, empty</code>	<code>ContentUnavailableView.search(text:)</code> – the system's stock no-results view
<code>.loaded(repos, isRefreshing)</code>	The <code>List</code> ; footer leaf shows the timestamp plus a mini-spinner while refreshing – results never blank mid-typing
<code>.failed(message)</code>	<code>ContentUnavailableView</code> with the message and a prominent Retry button

Navigation is value-based: rows are `NavigationLink(value: repo)` , and one `navigationDestination(for: Repo.self)` per stack declares where `Repo` values go. The search field disables autocapitalization/autocorrection and honors Return via `submitImmediately()` .

A real-world scar, kept on purpose. The error branch's accessibility identifier lives on the error *description text*, not the container. An earlier version put `search.errorView` on the whole `ContentUnavailableView` – and discovered the component re-parents a container identifier onto its merged children, clobbering the Retry button's own `search.retryButton` at runtime. The interim workaround (test queries matching the literal English "Something went wrong") then broke the moment the app grew a second language. The final shape – identifier on the one leaf that genuinely is the error message – is both correct and locale-independent, and the comment at the site tells the whole story.

RepoRowView

Small, stateless, preview-driven. Three details earn attention:

- **Locale-correct numbers** – the star count renders via `.formatted()` (67,000 – or 67.000 in German), not raw interpolation. The v1 `Label("\(count)")` also silently minted a meaningless bare `%lld` key into the String Catalog – an untranslatable entry and a collision magnet.
- **Dynamic Type that actually works** – the stats row is wrapped in `ViewThatFits(in: .horizontal)` with a vertical fallback, and the summary's two-line limit is lifted at accessibility text sizes

(`dynamicTypeSize.isAccessibilitySize`). The spec had claimed "Dynamic Type-friendly layouts" while the code contained zero Dynamic Type API – the audit's bluntest catch.

- **One VoiceOver sentence per row** – `.accessibilityElement(children: .combine)` plus a label built from *whole localized sentences* (four variants for the optional-field combinations), never fragments joined with `"", "`. Fragment-joining bakes in English word order and hands translators isolated pieces they can't reorder. The row's secondary text color is also slightly darkened from the system default – the automated accessibility audit measured stock `.secondary` at ~3.9:1 against WCAG AA's 4.5:1 floor for this size.

RepoDetailView – the view model that isn't

Displays a `Repo` it already holds, toggles a favorite, links out. No async orchestration, so no view model. Live favorite state comes from a *parameterized* `@Query` built in `init` (with the value-captured `#Predicate` idiom); the write goes through `FavoritesStore`. The toolbar button is the modern shape: `Button(title:systemImage:)` so the accessibility label derives from the title instead of drifting in a parallel modifier, `.contentTransition(.symbolEffect(.replace))` for the star swap, `.sensoryFeedback(.success, trigger:)` for the haptic, and `.accessibilityAddTraits(.isSelected)` when favorited so VoiceOver conveys *state*, not just a changing name. The GitHub link renders only for http(s) URLs – decoded URLs are otherwise unvalidated strings.

FavoritesView

Reads live via `@Query(sort: \.savedAt, order: .reverse)`. The empty state is an `.overlay` on the `List` rather than an if/else swap – keeping the List's identity means the last row's delete animates instead of jump-cutting to the empty screen. An `EditButton` in the toolbar provides a visible, non-gestural delete path (Switch Control and Voice Control users can't be expected to perform a precise swipe – and it made the delete path UI-testable). Multi-row deletes go through the store's batch API: one `context.save()`, not one per row.

10 Persistence: SwiftData, @Query, and the Store Split

`FavoriteRepo` is the `@Model`; reads use `@Query` directly in views (live, animated, no "refresh the list" bugs), writes go through `FavoritesStore` so the rules live in one unit-testable place. Details the audit sharpened:

- **.unique is a safety net, not the mechanism.** The comment on `@Attribute(.unique) var repoID` used to credit SwiftData upsert semantics for preventing duplicates. In truth the dedup this app relies on is the fetch-then-branch in `existingFavorite(for:)` – remove the attribute and every test still passes. The constraint remains as a store-level backstop, the comment now says exactly that, and a test inserts a duplicate directly to pin the actual observed behavior (the store collapses it to one row).
- **isFavorite throws.** The v1 signature returned `Bool` via `try?`, which conflated "the fetch failed" with "not favorited" – a store error rendered as an unfilled star. Distinguishing absence from failure is the caller's decision to make, not the API's to erase.

- **Batch remove.** `remove(_ favorites: some Sequence<FavoriteRepo>)` deletes N rows under a single `save()` – save granularity is precisely the kind of write rule the store exists to own.

Tests run against a fresh in-memory `ModelContainer` per test – hermetic, parallel-safe, no mocks. The UI tests use the same trick via a launch argument.

11 The Composition Root: Dependency Injection Without Magic

`AppDependencies`, built once by `RepoScoutApp`, decides every concrete implementation: live vs mock client, persistent vs in-memory store. Two corrections from the audit are worth internalizing:

- **Both test hooks are DEBUG-only.** v1 gated the mock-network flag with `#if DEBUG` but left `UITestInMemoryStore` honored in Release – a data-loss switch shipping in production builds. Symmetric gating now; a release binary ignores both.
- **No fatalError on container failure.** The most common real-world reason a `ModelContainer` fails to build is a schema-migration mismatch *after an app update* – and crash-on-launch is the worst possible response, because it's a crash loop the user cannot escape. The composition root now logs the failure and falls back to an in-memory store: degraded (favorites won't persist this session) but alive. The comment explains the reasoning; "the app has no meaningful degraded mode" turned out to be false the moment someone asked *when does this actually fail?*

Constructor injection for the view model, `.modelContainer` for `SwiftData` – the environment carries what the environment is for. UI tests flip the whole world with launch arguments; the release binary contains none of it.

12 The Concurrency Model: Swift 6, SE-0461, and What "async" Actually Means Now

This is the chapter the audit rewrote, and if you take one thing from this guide, take this. The project compiles in Swift 6 language mode with `SWIFT_DEFAULT_ACTOR_ISOLATION = MainActor` and `SWIFT_APPROACHABLE_CONCURRENCY = YES` – Xcode 26's defaults for new projects. The second setting enables **SE-0461 (NonisolatedNonsendingByDefault)**, which changes something fundamental:

The rule that changed. A plain `nonisolated async` function no longer hops to a background executor. It runs on its *caller's* executor. If a `MainActor` view model awaits your `nonisolated async` networking function, that function's body – including your JSON decoding – executes **on the main actor**. It compiles, it passes tests, and it drops frames. The `async` keyword never guaranteed "off the main thread"; under SE-0461 that folklore is now visibly, checkably false.

The first edition of this codebase had exactly that bug, wearing exactly that folklore as a doc comment ("decoded on a background URLSession task"). The fix is one attribute:

```
@concurrent
```

```
func searchRepositories(matching query: String) async throws -> [Repo]
```

`@concurrent` opts the function back out to the global concurrent executor: the request and the ~100KB decode genuinely leave the main actor, and `Repo: Sendable` is what makes the results' return trip legal. The doc comment on this function is now the concurrency lesson of the codebase.

Where the annotations live

With `MainActor` as the default, UI-facing code needs no annotations at all – views, the view model, `AppDependencies`, `FavoritesStore` are main-actor implicitly (which is also why the store's fetch-then-branch `toggle` has no race: two rapid taps are two serialized main-actor turns). `nonisolated` marks the types that cross boundaries – models, DTOs, the client – and `@concurrent` marks the one function that must *leave* its caller's executor. Actors appear where mutable state is genuinely shared: the mock client and the test doubles.

Type-checks vs. is-safe: the Combine caveat

The second thing v1's comments got wrong: the claim that default `MainActor` isolation "is why the Combine sinks can touch state without hopping." Isolation is why it *type-checks*. What makes it *true* is `scheduler: DispatchQueue.main` on the debounce and `on: .main` on the timer – Combine invokes sink closures on whatever thread the upstream delivers on and promises nothing. Swap that scheduler for a background queue and the code still compiles cleanly, then traps at runtime on the actor precondition (the audit verified this empirically). The codebase now marks those closures `@MainActor` explicitly and comments the scheduler arguments as load-bearing: the annotation makes the compiler check the assumption instead of silently inheriting it.

Pre-SwiftUI technique (c. 2019)	RepoScout does it with...
UIViewController + delegates/closures	SwiftUI View structs; state drives UI
KVO / NotificationCenter for change propagation	<code>@Observable</code> property tracking (per- <code>body</code> read attribution – see the footer-leaf lesson, §8)
GCD queues, <code>DispatchQueue.main.async</code>	<code>async/await</code> , actors, <code>MainActor</code> isolation checked at compile time – with <code>@concurrent</code> as the explicit "go off-main" verb
Core Data stack + <code>NSFetchedResultsController</code>	SwiftData <code>ModelContainer</code> + <code>@Query</code>
<code>NSTimer</code> you must invalidate	Combine <code>Timer.publish</code> with an owned <code>AnyCancellable</code> , started and stopped with the UI that needs it
OCMock-style mocking	Protocol conformances + actor spies; no mocking framework

13 Combine's Role in 2026 – the Honest Boundary

Combine appears in exactly two places, both *streams of events over time*: the debounced search pipeline and the scoped one-second ticker. Everything else – every one-shot request/response – is `async/await`; `LiveGitHubClient` contains no Combine at all, and says so.

The boundary rule: *if a value arrives once, `await` it; if values keep arriving, model them as a stream*. And the honest footnote, now with a sharper edge from Section 8: Combine's stream operators are not free of judgment – `removeDuplicates()` looked like exactly the right operator for the search pipeline and was quietly wrong for a non-deterministic operation. Operators encode assumptions; the lesson is to check the assumption, not just reach for the operator. `AsyncSequence` / `AsyncStream` (plus `swift-async-algorithms`' `debounce`) are the concurrency-native alternative a 2026 team would also evaluate; this app chose Combine deliberately – it ships in the OS, the needed operators are built in, and it remains ubiquitous in the codebases this app's readers will actually encounter. That's a choice about which ubiquitous tool to teach, not a claim that Combine is categorically right. `ARCHITECTURE.md` carries the fuller argument.

14 Unit Testing with Swift Testing

The unit suites use Swift Testing – `@Suite` structs, `@Test` functions, `#expect` / `#require`, parameterized cases – annotated `@MainActor` where they touch main-actor types (the test target does not inherit the app's default isolation). The supporting cast in `TestSupport.swift` is the suite's real teaching content:

- **Actor test doubles** – `SpyGitHubClient` (records queries), `GatedGitHubClient` (suspends every request on a continuation until the test says `open()` – letting a test hold the world still while it observes `.loading`), `KeyedGatedGitHubClient` (per-query gates, for supersession tests). All are cancellation-safe via `withTaskCancellationHandler` – a v1 gap: a cancelled task used to hang in the gate forever, which meant the cancellation paths *couldn't* be tested.
- **`poll(until:)` that actually fails** – v1's version recorded a timeout Issue and then *returned normally*, so a timed-out wait fell through into subsequent awaits and could hang the test instead of failing it. It now re-checks the condition once after the final sleep (closing a blind-spot race) and throws.
- **`StubURLProtocol`** – the canonical way to test a `URLSession` client offline: a stub protocol class injected via `URLSessionConfiguration.protocolClasses`, handler state guarded by a real `Mutex`. The v1 comment claimed `.serialized` test execution made an unsynchronized static safe – wrong actor, wrong race: the contention is test-thread vs. `URLSession`'s loader thread, which test ordering never addressed.

Coverage highlights: DTO decoding with inline fixtures and parameterized malformed payloads; the full HTTP-status error-mapping table (403 with and without rate-limit headers, 422, 2xx, garbage-200, `URLError(.cancelled)` → `CancellationError`, and one clean 200 round-trip); the supersession invariant (a cancelled search whose response *succeeds late* cannot clobber the newer result); debounce coalescing; failure-then-retype re-fires while success-then-retype doesn't; retry-after-failure recovers via a scripted

client; the ticker seeds `now` immediately; duplicate favorite inserts collapse to one row; batch remove saves once.

15 UI Testing: BDD Scenarios over XCTestCase Activities

Unchanged in shape from the first edition – `Given/When/Then` free functions wrapping `XCTestContext.runActivity` so reports read as scenarios; screen objects (`SearchScreen` , `RepoDetailScreen` , `FavoritesScreen`) encapsulating queries behind intentions; hermetic launches via DEBUG-only arguments. What the audit added:

- **New scenarios** – empty results (the previously untestable fourth state, via the `emptyResults` mock scenario), Retry-actually-recovers (via `searchErrorOnce` – the button is tapped, not just asserted to exist), and Edit-button-driven deletion in Favorites.
- **Keyboard-focus waits** before `typeText` – the single most common XCUITest CI flake, addressed rather than hoped away.
- **Automated accessibility audits** – `performAccessibilityAudit()` runs against the app's screens as part of the suite; it's the test that caught the contrast failure in Section 9, and it runs again at the largest accessibility text size with suppressions removed.
- **A launch-performance metric** in the launch test, so the smoke test asserts something.
- **Locale honesty** – with German added, the suite was re-run in German. Queries the app controls are locale-independent (identifiers, including the error-text identifier from Section 9). Four queries remain locale-bound because they match strings *Apple* owns and exposes no identifier for (the system "No Results" view, `EditButton`'s title, the delete confirmation, and the tab-bar button that SwiftUI's `Tab` API gives no identifier hook for). Those four are documented in the screen objects rather than papered over – a truthful limitation beats a brittle workaround.

16 Localization and Accessibility

A String Catalog with actual localizations

v1 shipped a catalog that was an extraction skeleton – 32 keys, zero translations, no plurals, no translator comments. It demonstrated the file format while skipping its point. The catalog now carries:

- **A complete German localization** – every key, with format specifiers preserved and reordered positionally where German word order demands it. German is a deliberately mean test: compound nouns stress layouts, and the plural rules differ from English.
- **Plural variations** – "Updated 1 second ago" / "Updated 42 seconds ago" (the string was changed from "42s" to a spelled-out noun specifically so inflecting languages have a noun to inflect).
- **Translator comments** – every `String(localized:)` call site passes `comment:` , and catalog-only keys carry comments, because "Search" with no context could be a verb, a tab name, or a heading, and a

translator cannot guess.

- **No junk keys** – the bare `%lld` entry (a raw interpolated integer used as a localization key: untranslatable, and a collision magnet) is gone; numbers are formatted as *values* via `.formatted()`, not localized as keys.

Accessibility as architecture

- **Rows are single VoiceOver elements** speaking whole localized sentences – the app's best lesson, now documented (it wasn't, in v1's own docs).
- **Dynamic Type actually accommodated** – `ViewThatFits` reflow, accessibility-size line-limit relaxation.
- **Contrast measured, not assumed** – the audit test flagged stock `.secondary` at caption sizes below WCAG AA; the row text is darkened with a comment explaining the measurement.
- **State conveyed as state** – the favorite toggle carries `.isSelected`, not just a changing label.
- **Ambient noise silenced** – the once-per-second footer is hidden from VoiceOver entirely.
- **Identifiers ≠ accessibility** – a correction to v1's framing: `accessibilityIdentifier` is a test-automation hook, invisible to assistive technology. The accessibility work is the labels, traits, and layout behavior; the identifiers are the UI tests' contract. The docs no longer conflate them.

17 Shipping Hygiene: the Parts That Aren't Code

"Professional" is not only a statement about code, and this section exists because the audit's harshest findings were all outside the Swift. What the repo now carries, and why each matters:

Artifact	Why it's there
Shared scheme (xcsharedata/xcschemes/)	Auto-created schemes are per-user and untracked; without a shared one, every documented <code>xcodebuild</code> command fails on a fresh clone. The #1 reproducibility bug in shared Xcode projects.
<code>LICENSE</code> (MIT)	A public repo with no license is all-rights-reserved by default – readers would be legally barred from copying the patterns the app exists to teach.
<code>PrivacyInfo.xcprivacy</code>	Required for App Store submission since 2024. This app's manifest is the most instructive kind: tracking false, every array empty – "we collect nothing," declared explicitly rather than implied by absence.
Real app icon + accent color	Light/dark/tinted 1024pt variants and a defined any/dark accent pair – replacing empty template stubs the README had (falsely) described as done.
<code>ITSAppUsesNonExemptEncryption = NO</code>	One line that removes a manual export-compliance question from every TestFlight/App Store upload. Its absence is a reliable tell a project has never been through one.
App category (Developer Tools)	Required by App Store Connect; set in the project so the archive isn't flagged.
ATS: fully strict, no exceptions	Deliberate and verified – <code>api.github.com</code> meets default App Transport Security, so no exception dictionary exists. Many sample apps blanket-disable ATS "to make it work"; not doing so is the lesson.
<code>TARGETED_DEVICE_FAMILY = 1</code> (iPhone)	v1 declared iPad support while explicitly not designing for it – which also exposed a real bug class (two iPad windows sharing one view model's state). Scoping the binary to what was actually designed is the honest version.
No committed <code>DEVELOPMENT_TEAM</code>	A personal team ID doesn't belong in a public repo – it leaks the ID and breaks signing for everyone else.
<code>docs/process-archive/</code>	The original build spec/plan, kept for provenance but headed with a disclaimer – the plan's code listings predate the audit fixes and must not be used as reference.

18 Design Decisions, Trade-offs, and Known Edges

Decisions worth restating

- **A view model only where it earns its keep.** One screen has orchestration logic; one screen has a view model.
- **State as a closed enum, including the state everyone forgets** – stale-while-revalidate is representable, so refinements don't blank the screen.

- **The protocol seam is the testing strategy.** Unit tests, UI tests, and previews all substitute at the same joint. No mocking framework, none missed.
- **Comments must be true under the actual build settings.** The audit's deepest finding was not a crash – it was doc comments confidently describing pre-SE-0461 semantics. In a teaching codebase, a wrong comment is a worse defect than a slow decode.
- **Errors are typed at the boundary, discriminated honestly (403 ≠ rate-limited), translated once, displayed as friendly copy.**
- **Determinism is designed** – gated continuations, scripted scenarios, fixture clients, in-memory stores.

Known edges (deliberate, documented)

- **No pagination, auth, image loading, or iPad layout** – out of scope by design; each would dilute the teaching signal with plumbing. The device family now matches that scope.
- **Four UI-test queries are locale-bound to Apple-owned strings** (system no-results view, EditButton, delete confirmation, tab-bar button) – SwiftUI exposes no identifier hooks for them; the screen objects document it.
- **The debounce negative-assertion is bounded** – "no re-fire within the window" can't prove "never"; the test comment says so.
- **Combine remains a curriculum choice.** `AsyncStream` + `async-algorithms` is the equally-valid 2026 alternative; the app teaches the tool its readers will most often inherit, and says so.

Where to go next

Read `ARCHITECTURE.md` for the guided tour in prose form, then open the code in this order:

`Support/LoadState.swift` → `ViewModels/SearchViewModel.swift` → `Services/LiveGitHubClient.swift` (the `@concurrent` comment especially) → `Views/Search/SearchView.swift` (through `LastRefreshedFooter`) → `testExampleTests/SearchViewModelTests.swift`. The doc comments are the second half of this guide – they were written, and audited, to be read.